CNN for Galaxy Morphology + Webapp

Student id: C00313459

Name: Amisha Das

1. Introduction

This is a basic CNN made from scratch to predict a galaxy's type. At the end there is also a webapp that the user can use to input an image and get its predicted type.

Data Source

- Data can also be downloaded from here or the following cell as it is a large data set

```
# !pip install astroNN
# from astroNN.datasets import galaxy10
# galaxy10.load_data()

import h5py

file_path="Galaxy10.h5"
```

2. Data

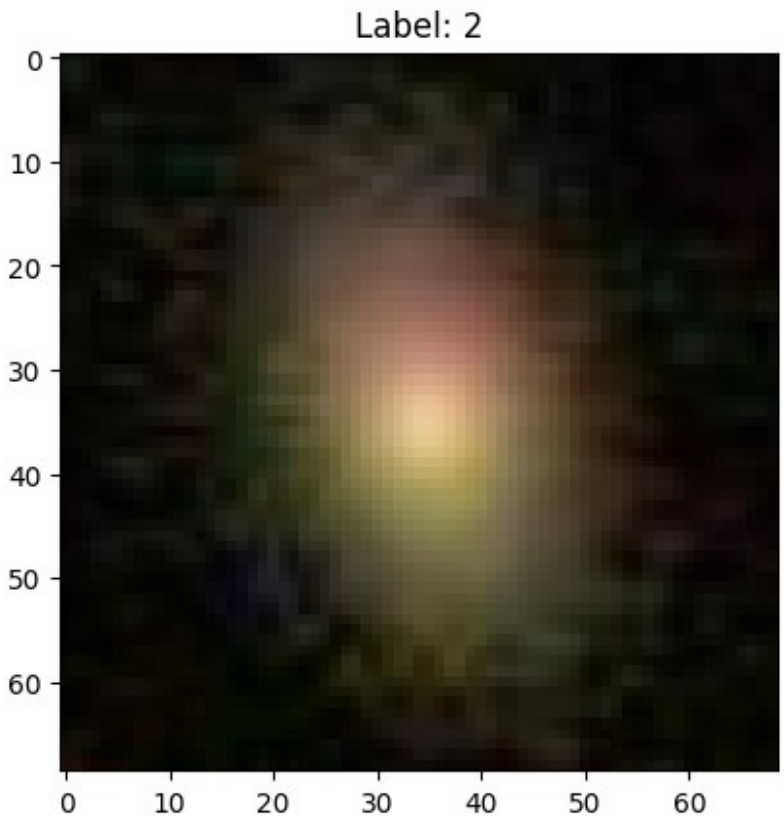
```
with h5py.File(file_path, 'r') as f:
    print(f.keys())
    images = f['images'][:].astype('float32') / 255.0
    labels = f['ans'][:]

print("Image Shape:", images.shape)
print("Labels Shape:", labels.shape)
```

```
<KeysViewHDF5 ['ans', 'images']>
Image Shape: (21785, 69, 69, 3)
Labels Shape: (21785,)
```

```
import matplotlib.pyplot as plt
import numpy as np

plt.imshow(images[0])
plt.title(f"Label: {labels[0]}")
plt.show()
```



2.2. Preparing the data

```
from sklearn.model_selection import train_test_split
from tensorflow.keras.utils import to_categorical
X = (images / 255.0).astype('float32')
y = to_categorical(labels, num_classes=10)

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
```

3 CNN model

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten,
Dense, Dropout, BatchNormalization

model = Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=(69, 69, 3)),
    BatchNormalization(),
    MaxPooling2D((2,2)),
```

```
    Conv2D(64, (3,3), activation='relu'),
    BatchNormalization(),
    MaxPooling2D((2,2)),

    Conv2D(128, (3,3), activation='relu'),
    BatchNormalization(),
    MaxPooling2D((2,2)),

    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.4),
    Dense(10, activation='softmax')
])

model.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])
model.summary()
```

```
Model: "sequential_1"
```

```
┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━
┓
┃ Layer (type)                         ┃ Output Shape                ┃
Param # ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━
┩
│ conv2d_2 (Conv2D)                    │ (None, 67, 67, 32)          │
896 │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────
┤
│ batch_normalization                  │ (None, 67, 67, 32)          │
128 │
│ (BatchNormalization)                 │                             │
│
├──────────────────────────────────────┼─────────────────────────────┼─────────────────
┤
│ max_pooling2d_2 (MaxPooling2D)       │ (None, 33, 33, 32)          │
0 │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────
┤
│ conv2d_3 (Conv2D)                    │ (None, 31, 31, 64)          │
18,496 │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────
┤
│ batch_normalization_1                │ (None, 31, 31, 64)          │
256 │
│ (BatchNormalization)                 │                             │
│
├──────────────────────────────────────┼─────────────────────────────┼─────────────────
┤
```

```
| max_pooling2d_3 (MaxPooling2D)       | (None, 15, 15, 64)          |
0 |
├──────────────────────────────────────┼─────────────────────────────┼
──────────────────┤
| conv2d_4 (Conv2D)                    | (None, 13, 13, 128)         |
73,856 |
├──────────────────────────────────────┼─────────────────────────────┼
──────────────────┤
| batch_normalization_2                | (None, 13, 13, 128)         |
512 |
| (BatchNormalization)                 |                             |
|
├──────────────────────────────────────┼─────────────────────────────┼
──────────────────┤
| max_pooling2d_4 (MaxPooling2D)       | (None, 6, 6, 128)           |
0 |
├──────────────────────────────────────┼─────────────────────────────┼
──────────────────┤
| flatten_1 (Flatten)                  | (None, 4608)                |
0 |
├──────────────────────────────────────┼─────────────────────────────┼
──────────────────┤
| dense_2 (Dense)                      | (None, 128)                 |
589,952 |
├──────────────────────────────────────┼─────────────────────────────┼
──────────────────┤
| dropout_1 (Dropout)                  | (None, 128)                 |
0 |
├──────────────────────────────────────┼─────────────────────────────┼
──────────────────┤
| dense_3 (Dense)                      | (None, 10)                  |
1,290 |
└──────────────────────────────────────┴─────────────────────────────┴
──────────────────┘

 Total params: 685,386 (2.61 MB)

 Trainable params: 684,938 (2.61 MB)

 Non-trainable params: 448 (1.75 KB)
```

4. Training

```
from tensorflow.keras.callbacks import EarlyStopping,
ReduceLROnPlateau

callbacks = [
    EarlyStopping(patience=5, restore_best_weights=True),
```

```
    ReduceLROnPlateau(patience=3, factor=0.2, verbose=1)
]

history = model.fit(X_train, y_train, epochs=30, validation_split=0.2,
callbacks=callbacks)
```

```
Epoch 1/30
436/436 ━━━━━━━━━━━━━━━━━━━━ 26s 54ms/step - accuracy: 0.3788 - loss:
1.6642 - val_accuracy: 0.3144 - val_loss: 1.8702 - learning_rate:
0.0010
Epoch 2/30
436/436 ━━━━━━━━━━━━━━━━━━━━ 23s 54ms/step - accuracy: 0.5221 - loss:
1.2125 - val_accuracy: 0.1773 - val_loss: 2.6642 - learning_rate:
0.0010
Epoch 3/30
436/436 ━━━━━━━━━━━━━━━━━━━━ 23s 52ms/step - accuracy: 0.5569 - loss:
1.0967 - val_accuracy: 0.0290 - val_loss: 6.3661 - learning_rate:
0.0010
Epoch 4/30
435/436 ━━━━━━━━━━━━━━━━━━━━ 0s 50ms/step - accuracy: 0.6253 - loss:
0.9847
Epoch 4: ReduceLROnPlateau reducing learning rate to
0.00020000000949949026.
436/436 ━━━━━━━━━━━━━━━━━━━━ 23s 53ms/step - accuracy: 0.6253 - loss:
0.9847 - val_accuracy: 0.0720 - val_loss: 14.2808 - learning_rate:
0.0010
Epoch 5/30
436/436 ━━━━━━━━━━━━━━━━━━━━ 23s 52ms/step - accuracy: 0.6690 - loss:
0.8437 - val_accuracy: 0.6314 - val_loss: 0.9096 - learning_rate:
2.0000e-04
Epoch 6/30
436/436 ━━━━━━━━━━━━━━━━━━━━ 23s 53ms/step - accuracy: 0.7050 - loss:
0.8020 - val_accuracy: 0.1357 - val_loss: 3.7166 - learning_rate:
2.0000e-04
Epoch 7/30
436/436 ━━━━━━━━━━━━━━━━━━━━ 23s 52ms/step - accuracy: 0.7253 - loss:
0.7461 - val_accuracy: 0.1867 - val_loss: 6.0126 - learning_rate:
2.0000e-04
Epoch 8/30
435/436 ━━━━━━━━━━━━━━━━━━━━ 0s 51ms/step - accuracy: 0.7375 - loss:
0.7293
Epoch 8: ReduceLROnPlateau reducing learning rate to
4.0000001899898055e-05.
436/436 ━━━━━━━━━━━━━━━━━━━━ 24s 54ms/step - accuracy: 0.7375 - loss:
0.7293 - val_accuracy: 0.2341 - val_loss: 2.9499 - learning_rate:
2.0000e-04
Epoch 9/30
436/436 ━━━━━━━━━━━━━━━━━━━━ 24s 54ms/step - accuracy: 0.7571 - loss:
0.6678 - val_accuracy: 0.4260 - val_loss: 1.4786 - learning_rate:
4.0000e-05
```

```
Epoch 10/30
436/436 ━━━━━━━━━━━━━━━━━━━━ 24s 54ms/step - accuracy: 0.7582 - loss:
0.6636 - val_accuracy: 0.6262 - val_loss: 0.8763 - learning_rate:
4.0000e-05
Epoch 11/30
436/436 ━━━━━━━━━━━━━━━━━━━━ 23s 54ms/step - accuracy: 0.7611 - loss:
0.6469 - val_accuracy: 0.6885 - val_loss: 0.7957 - learning_rate:
4.0000e-05
Epoch 12/30
436/436 ━━━━━━━━━━━━━━━━━━━━ 23s 52ms/step - accuracy: 0.7577 - loss:
0.6636 - val_accuracy: 0.7163 - val_loss: 0.7908 - learning_rate:
4.0000e-05
Epoch 13/30
436/436 ━━━━━━━━━━━━━━━━━━━━ 23s 53ms/step - accuracy: 0.7652 - loss:
0.6377 - val_accuracy: 0.2903 - val_loss: 2.2126 - learning_rate:
4.0000e-05
Epoch 14/30
436/436 ━━━━━━━━━━━━━━━━━━━━ 23s 54ms/step - accuracy: 0.7603 - loss:
0.6435 - val_accuracy: 0.5608 - val_loss: 1.4122 - learning_rate:
4.0000e-05
Epoch 15/30
436/436 ━━━━━━━━━━━━━━━━━━━━ 0s 50ms/step - accuracy: 0.7691 - loss:
0.6352
Epoch 15: ReduceLROnPlateau reducing learning rate to
8.000000525498762e-06.
436/436 ━━━━━━━━━━━━━━━━━━━━ 23s 53ms/step - accuracy: 0.7691 - loss:
0.6352 - val_accuracy: 0.5273 - val_loss: 1.5090 - learning_rate:
4.0000e-05
Epoch 16/30
436/436 ━━━━━━━━━━━━━━━━━━━━ 23s 53ms/step - accuracy: 0.7760 - loss:
0.6197 - val_accuracy: 0.7854 - val_loss: 0.6245 - learning_rate:
8.0000e-06
Epoch 17/30
436/436 ━━━━━━━━━━━━━━━━━━━━ 23s 53ms/step - accuracy: 0.7660 - loss:
0.6259 - val_accuracy: 0.7814 - val_loss: 0.6268 - learning_rate:
8.0000e-06
Epoch 18/30
436/436 ━━━━━━━━━━━━━━━━━━━━ 23s 53ms/step - accuracy: 0.7689 - loss:
0.6247 - val_accuracy: 0.7372 - val_loss: 0.7056 - learning_rate:
8.0000e-06
Epoch 19/30
436/436 ━━━━━━━━━━━━━━━━━━━━ 0s 52ms/step - accuracy: 0.7685 - loss:
0.6115
Epoch 19: ReduceLROnPlateau reducing learning rate to
1.6000001778593287e-06.
436/436 ━━━━━━━━━━━━━━━━━━━━ 24s 55ms/step - accuracy: 0.7685 - loss:
0.6115 - val_accuracy: 0.7866 - val_loss: 0.6252 - learning_rate:
8.0000e-06
Epoch 20/30
```

```
436/436 ━━━━━━━━━━━━━━━━━━━━ 25s 58ms/step - accuracy: 0.7708 - loss:
0.6165 - val_accuracy: 0.7926 - val_loss: 0.5834 - learning_rate:
1.6000e-06
Epoch 21/30
436/436 ━━━━━━━━━━━━━━━━━━━━ 24s 55ms/step - accuracy: 0.7880 - loss:
0.6039 - val_accuracy: 0.7943 - val_loss: 0.5880 - learning_rate:
1.6000e-06
Epoch 22/30
436/436 ━━━━━━━━━━━━━━━━━━━━ 24s 54ms/step - accuracy: 0.7721 - loss:
0.6250 - val_accuracy: 0.7900 - val_loss: 0.5834 - learning_rate:
1.6000e-06
Epoch 23/30
435/436 ━━━━━━━━━━━━━━━━━━━━ 0s 51ms/step - accuracy: 0.7727 - loss:
0.6171
Epoch 23: ReduceLROnPlateau reducing learning rate to
3.200000264769187e-07.
436/436 ━━━━━━━━━━━━━━━━━━━━ 23s 54ms/step - accuracy: 0.7727 - loss:
0.6171 - val_accuracy: 0.7915 - val_loss: 0.5880 - learning_rate:
1.6000e-06
Epoch 24/30
436/436 ━━━━━━━━━━━━━━━━━━━━ 25s 57ms/step - accuracy: 0.7763 - loss:
0.6063 - val_accuracy: 0.7929 - val_loss: 0.5837 - learning_rate:
3.2000e-07
Epoch 25/30
436/436 ━━━━━━━━━━━━━━━━━━━━ 25s 58ms/step - accuracy: 0.7773 - loss:
0.6091 - val_accuracy: 0.7909 - val_loss: 0.5827 - learning_rate:
3.2000e-07
Epoch 26/30
436/436 ━━━━━━━━━━━━━━━━━━━━ 25s 57ms/step - accuracy: 0.7806 - loss:
0.6104 - val_accuracy: 0.7935 - val_loss: 0.5828 - learning_rate:
3.2000e-07
Epoch 27/30
436/436 ━━━━━━━━━━━━━━━━━━━━ 24s 56ms/step - accuracy: 0.7813 - loss:
0.6165 - val_accuracy: 0.7926 - val_loss: 0.5826 - learning_rate:
3.2000e-07
Epoch 28/30
436/436 ━━━━━━━━━━━━━━━━━━━━ 24s 55ms/step - accuracy: 0.7722 - loss:
0.6258 - val_accuracy: 0.7929 - val_loss: 0.5828 - learning_rate:
3.2000e-07
Epoch 29/30
436/436 ━━━━━━━━━━━━━━━━━━━━ 24s 55ms/step - accuracy: 0.7767 - loss:
0.6058 - val_accuracy: 0.7923 - val_loss: 0.5827 - learning_rate:
3.2000e-07
Epoch 30/30
435/436 ━━━━━━━━━━━━━━━━━━━━ 0s 53ms/step - accuracy: 0.7729 - loss:
0.6186
Epoch 30: ReduceLROnPlateau reducing learning rate to
6.400000529538374e-08.
436/436 ━━━━━━━━━━━━━━━━━━━━ 24s 55ms/step - accuracy: 0.7729 - loss:
```

```
0.6185 - val_accuracy: 0.7926 - val_loss: 0.5831 - learning_rate:
3.2000e-07
```

5. Evaluating

```
test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test Accuracy:", test_acc)

137/137 ━━━━━━━━━━━━━━━━━━━━ 2s 12ms/step - accuracy: 0.7864 - loss:
0.6127
Test Accuracy: 0.787468433380127

model.save("galaxy_cnn_model.h5")

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.
```

6. WebApp

The accompanying webapp based on this model can be found [here](https://cnnapp-amisha.streamlit.app/](https://cnnapp-amisha.streamlit.app/).

Appendices, Acknowledgments and References

- https://github.com/henrysky/astroNN
- https://www.kaggle.com/code/amydas/basic-cnn-to-classify-galaxy-morphology
- ChatGPT for debugging